DFSha Fase II

Implementación y validación grupal

Sistemas Distribuidos 2026 2

Opción 1 cliente servidor con servicio S2S

Lifeng Chen

Santiago Lopez Angel

Arquitectura	3 MDS y 4 DataNodes
Replicación	R 3 y W 2 con cadena S2S
Consistencia	Raft mínimo, leases y snapshots
Validación	9 pruebas, Docker con TLS y 3 VMs en AWS

Informe técnico final

Sistemas Distribuidos 2026 2

Integrantes: Lifeng Chen y Santiago Lopez Angel

Resumen

Se implementó DFSha como un sistema de archivos distribuido de Opción 1. Tres Metadata Servers coordinan el namespace, los leases y las versiones mediante una implementación mínima de Raft. Cuatro DataNodes almacenan bloques inmutables cifrados. El cliente obtiene ubicaciones desde el MDS, divide cada archivo y envía cada bloque una sola vez al primer nodo de una cadena S2S. La publicación requiere dos réplicas verificadas por bloque y un commit de metadatos confirmado por mayoría. La base y su validación se prepararon con asistencia de Codex; los integrantes deben revisar y defender las decisiones y registrar su participación real.

La implementación cubre RF1, RF2 y la semántica declarada de RF3. El 6 de noviembre de 2026 pasaron nueve pruebas locales en 40.87 segundos. Se ejecutaron además Docker Compose con TLS en GitHub Actions y un despliegue Docker en tres VMs de AWS Academy. En AWS se recuperó un archivo de 16777236 bytes repartido en 17 bloques, se mantuvo la integridad con un DataNode detenido, se confirmó una escritura con réplica desconectada, se restauró la redundancia y se comprobó relevo de líder. Sin mayoría se rechazaron las escrituras. Las mediciones locales, Docker y AWS se presentan separadas.

Requisitos y alcance

RF1 se implementa con **ls**, **cd**, **mkdir**, **rmdir**, **rm** y **stat**. Cada usuario trabaja en **/users/{uid}** y el servidor rechaza rutas de otro usuario. **cd** es estado local del cliente y valida el destino con **stat**.

RF2 se implementa con **put** y **get**. **put** abre una sesión, divide el archivo, asigna pipelines, carga bloques y confirma un manifiesto. **get** fija una versión, descarga bloques en paralelo, prueba réplicas alternativas y valida el SHA 256 final.

RF3 se implementa con **open**, **close**, **read**, **write**, **append** y **lock**. **write** admite creación, reemplazo, anexado y modificación parcial mediante copy on write. No admite huecos más allá de EOF ni semántica POSIX completa. La atomicidad visible corresponde al commit de una versión completa.

Comparación de propuestas

Los dos diseños individuales coincidieron en la separación entre control y datos, tres MDS con Raft, DataNodes con bloques inmutables, réplicas R igual a 3, confirmación W igual a 2, leases exclusivos y cifrado. Santiago propuso un pipeline secuencial para la escritura. Lifeng propuso que un DataNode de entrada replicara en paralelo.

Tema	Santiago	Lifeng	Arquitectura grupal
Escritura S2S	Cadena DN 1 a DN 2 a DN 3	Fan out desde el DN de entrada	Cadena para carga y copias directas para reparación
Control	3 o 5 MDS Raft	3 MDS Raft	3 MDS con núcleo Raft verificable
Réplicas	R 3, mínimo 2	R 3, W 2	R 3, commit con 2 verificadas
Consistencia	Metadatos linealizables y close to open	Manifiesto atómico y snapshots	Log por mayoría, snapshots y close to open
Seguridad	TLS, mTLS, Argon2id y AES GCM	TLS, mTLS, Argon2id y AES GCM	TLS, HMAC interno, scrypt y AES GCM

La cadena reduce el ancho de banda de subida del cliente y muestra comunicación S2S directa. La reparación S2S directa evita que el orden original del pipeline limite la recuperación. La sustitución de mTLS por TLS más HMAC y de Argon2id por scrypt se declara como una decisión de prototipo, no como equivalencia de seguridad completa.

Arquitectura

El cliente es una CLI y biblioteca Python. Descubre el líder consultando la lista de MDS, mantiene el directorio de trabajo, maneja tokens y descarga bloques concurrentemente. Ninguna dirección de DataNode está codificada en el cliente.

Cada MDS conserva dos bases SQLite independientes: una para el log y estado Raft, y otra para usuarios, directorios, archivos, versiones y leases. El algoritmo persiste términos y votos, ejecuta elecciones, replica entradas con **AppendEntries**, exige mayoría y aplica comandos en orden. Un seguidor puede convertirse en líder cuando el anterior deja de responder.

Cada DataNode usa un volumen propio. Los bytes se cifran con AES 256 GCM y se asocian mediante AAD al block id y la generación. El archivo de metadatos del bloque conserva longitud y SHA 256. El nodo escribe archivos temporales, hace **fsync** y renombra antes de responder.

Docker Compose contiene tres MDS y cuatro DataNodes en una red bridge privada y fue validado en GitHub Actions. En AWS se ejecutaron contenedores Docker con red de host sobre tres VMs t3.micro de Amazon Linux 2023. Los puertos de servicio solo admiten tráfico desde el grupo de seguridad del proyecto. La administración utiliza Systems Manager, sin abrir SSH. Los datos se guardan en directorios independientes sobre discos gp3 cifrados; los servicios usan TLS con una CA de laboratorio y SAN para las IP privadas.

Máquina	Zona AWS	Dirección privada	Servicios
VM 1	us-east-1a	172.31.22.6	MDS 1 en 8443 y DN 1 en 9443
VM 2	us-east-1b	172.31.34.173	MDS 2 en 8443 y DN 2 en 9443
VM 3	us-east-1c	172.31.10.22	MDS 3 en 8443, DN 3 en 9443 y DN 4 en 9444

La tercera VM contiene dos DataNodes: tres réplicas de servicio no equivalen necesariamente a tres máquinas o zonas distintas. Las IP públicas asignadas a las VMs solo permiten salida para instalar dependencias y contactar con Systems Manager; no se habilitó entrada pública a los servicios.

Distribución y publicación

El tamaño predeterminado es 8 MiB y se configura por variable de entorno. Las pruebas usan 64 KiB para forzar varios bloques con archivos pequeños. Para cada índice, el MDS rota la lista de DataNodes sanos y entrega hasta tres destinos. El token de bloque contiene operación, usuario, block id, generación, pipeline y expiración.

El primer DataNode valida el token, cifra y persiste el bloque y lo reenvía al siguiente. La respuesta contiene las réplicas que completaron el almacenamiento. Antes del commit, el MDS consulta cada nodo y compara longitud y SHA 256 con el manifiesto propuesto. Si un bloque tiene menos de dos réplicas, la versión no se publica.

El commit contiene la versión base, el epoch, el tamaño, el hash final y la lista ordenada de bloques. El líder lo incorpora al log y espera mayoría. La máquina de estados inserta el nuevo manifiesto, incrementa la versión y libera el lease en una sola transacción. La recuperación explícita del resultado por request id se implementa para mkdir; no se garantiza idempotencia general de todos los reintentos de commit.

Consistencia y concurrencia

Los metadatos confirmados se ordenan en el log. Un nodo sin mayoría no puede aceptar mutaciones. Las lecturas de archivos usan snapshots de manifiesto. Los bloques publicados no se modifican; una escritura parcial reconstruye el contenido afectado y carga bloques nuevos.

La exclusión es por archivo. El lease EX incluye holder, epoch, expiración y versión base. El segundo escritor recibe **LOCKED**. Si el primer cliente se pausa hasta que vence el lease, el siguiente obtiene un epoch mayor. El commit tardío falla porque el epoch o la versión base ya no coincide. Este fencing evita que un escritor zombi publique después de perder el lock.

Los lectores normales no toman lock y mantienen la versión fijada al abrir. Un commit posterior no modifica su manifiesto. En la operación normal las nuevas aperturas observan la versión confirmada. No se ofrece linealizabilidad de lectura bajo cualquier partición: falta ReadIndex para impedir que un líder aislado atienda una lectura atrasada.

Se corrigieron dos carreras: el cliente obtiene el lease antes de leer para modificar o anexar, y el líder serializa la planificación con la confirmación de la adquisición del lease. Se probaron dos clientes lanzados simultáneamente durante cinco rondas; en cada ronda hubo un solo ganador y un rechazo LOCKED.

Tolerancia a fallos

El MDS sondea los DataNodes antes de asignar bloques. Si un nodo no responde, lo excluye. Durante lectura, el cliente prueba las réplicas restantes y valida cada respuesta.

Un worker del líder revisa los manifiestos. Si faltan réplicas, elige una fuente sana y pide una copia S2S hacia un destino disponible. Después confirma el nuevo conjunto de ubicaciones mediante Raft. La versión lógica y el hash del archivo no cambian.

Si el líder MDS cae, los seguidores incrementan el término, votan y eligen otro líder. Los commits que ya alcanzaron mayoría permanecen. La prueba detuvo el líder real, descubrió otro, leyó el archivo y confirmó un directorio nuevo.

Un bloque parcial no se publica porque el DataNode solo responde después de cifrar, sincronizar y renombrar. El MDS tampoco acepta un manifiesto sin dos verificaciones por bloque. La recolección física de temporales y versiones antiguas se dejó como trabajo pendiente.

Seguridad

Las contraseñas se almacenan con scrypt y sal aleatoria. El MDS emite tokens de acceso HMAC de 15 minutos. Los DataNodes no aceptan el token general del usuario; requieren un token firmado y limitado al bloque y la operación.

La normalización de rutas impide salir de **/users/{uid}**. Las APIs internas requieren un secreto diferente del JWT. Docker activa TLS en MDS y DataNodes. Los bloques se cifran con AES 256 GCM y el cliente valida SHA 256 de bloque y archivo.

En AWS se generaron secretos, contraseña y claves por nodo aleatorios, conservados fuera del repositorio. La CA es de laboratorio. El prototipo no implementa mTLS por nodo ni un KMS; estas limitaciones están registradas. El grupo de seguridad solo permite los puertos privados 8443, 9443 y 9444 entre miembros del propio grupo.

APIs

El Metadata Service ofrece autenticación, namespace, apertura, asignación, commit, renovación, cierre y localización. El Data Service ofrece escritura encadenada, lectura, estado y reparación. HTTP se eligió para mantener contratos visibles y pruebas sencillas; la transferencia usa bytes y no JSON.

Los errores principales son **NOT_LEADER**, **NO_QUORUM**, **LOCKED**, **STALE_FENCING_TOKEN**, **VERSION_CONFLICT**, **INSUFFICIENT_REPLICAS**, **PERMISSION_DENIED** y **BLOCK_UNAVAILABLE**. El cliente sigue la pista del líder, reintenta otras réplicas o devuelve el error según el caso.

Pruebas y resultados

La suite ampliada levantó tres MDS y cuatro DataNodes con directorios aislados y puertos libres. Se ejecutó en Windows el 6 de noviembre de 2026. La evidencia XML conserva nombres, resultado y duración; GitHub Actions repitió la suite sobre Linux.

Prueba	Resultado observado
Carga y recuperación	Archivo dividido en al menos cuatro bloques, tres réplicas por bloque y bytes idénticos
RF3	Lectura por rango, modificación y anexado correctos
Concurrencia	Segundo escritor rechazado con **LOCKED**
Seguridad	Ruta ajena rechazada y bloques sin texto plano visible
Falla de DN	Lectura correcta y sustitución de la réplica caída
Falla de MDS	Nuevo líder, lectura y commit posterior
Transferencia interrumpida	El contenido anterior se conserva y el commit incompleto se rechaza
Corrupción	Una réplica cifrada alterada falla en lectura; el cliente obtiene otra copia íntegra
TLS	Se acepta la CA configurada y se rechaza una CA desconocida
Escritores simultáneos	Un ganador y un LOCKED en cada una de cinco rondas
Suite completa ampliada	9 pruebas aprobadas en 40.87 segundos

El benchmark local repitió cada condición tres veces:

Tamaño	Put mediana	Put MiB por segundo	Get mediana	Get MiB por segundo
64 KiB	0.313588 s	0.199	0.023220 s	2.692
512 KiB	0.644096 s	0.776	0.025934 s	19.280

El archivo mayor obtiene mejor throughput porque amortiza solicitudes, cifrado y commit. No se extrapolan estas cifras a AWS.

Resultados del despliegue AWS

Se configuró bloque de 1 MiB, R=3, W=2 y lease de 300 segundos. El archivo de 16777236 bytes produjo 17 bloques. La carga tardó 3.099673 segundos. El SHA-256 del original y de las recuperaciones fue c574fc22903d83367ba55623914d6bceffd407af241d20d001e3ec58edd1bf2e.

Tamaño AWS	Put mediana	Put MiB/s	Get mediana	Get MiB/s
1 MiB	0.285585 s	3.502	0.041090 s	24.337
8 MiB	1.364646 s	5.862	0.152955 s	52.303

Cada condición se repitió tres veces con verificación del hash. No se calcularon p95 ni capacidad máxima. Los tamaños y el hardware difieren de las pruebas locales; esta tabla no aísla el efecto de TLS o de la red.

Fase AWS	Resultado observado
baseline	RF1-RF3, TLS, autorización y archivo grande íntegro con tres réplicas
degraded	Lectura íntegra y nueva escritura con dn-1 detenido
recovered	Recuperación de tres réplicas y del hash original
failover	El líder cambia de VM 2 a VM 1 y confirma un mkdir nuevo
no-quorum	Con dos MDS detenidos se rechazan escrituras con NO_LEADER
concurrency	Cinco rondas simultáneas con un solo escritor aceptado

Los JSON de evidence/aws conservan la topología, hashes, controles y tiempos. Las fallas se inyectaron deteniendo contenedores; no se simuló la caída de una zona AWS completa. Los servicios se restauraron después de cada prueba y las VMs se detuvieron al finalizar para reducir consumo.

Limitaciones

La implementación Raft no contiene snapshots, compactación, pre vote ni membresía dinámica. Los DataNodes se configuran en el servidor, aunque el cliente recibe ubicaciones dinámicas. No existen cuotas, **mv**, garbage collection ni locks por rango. La reparación usa sondeo periódico. TLS autentica servidores y el secreto HMAC protege llamadas internas, pero no existe mTLS individual.

No existe ReadIndex para lecturas linealizables durante particiones, scrubbing de corrupción latente ni reparación paralela. La corrupción se detecta al leer y habilita una réplica alternativa; el worker de reparación comprueba metadatos. El cliente reconstruye archivos completos en memoria. La aplicación no ofrece un sistema de archivos de producción ni idempotencia general de todas las mutaciones. Las simplificaciones requieren revisión y, cuando corresponda, aprobación del profesor.

Contribuciones y defensa

El repositorio incluye un plan para que Lifeng y Santiago generen contribuciones trazables. Santiago puede extender el plano de control con heartbeat, membresía y pruebas de partición. Lifeng puede implementar garbage collection, corrupción inyectada y benchmark en AWS. Cada cambio debe entrar por una rama y un pull request revisado por el otro integrante.

Ambos deben poder explicar el flujo completo, el criterio de commit, los leases, el fencing, la lectura alternativa, la reparación y las limitaciones. El historial de Git no reemplaza esa comprensión.

Conclusión

DFSha demuestra una arquitectura distribuida verificable: separa control y datos, distribuye bloques, replica por S2S, publica versiones atómicas, coordina escritores y continúa ante la caída comprobada de un DataNode o del líder MDS. Las pruebas se ejecutaron en procesos locales, Docker Compose y tres VMs de AWS Academy. La entrega incluye código, instrucciones y evidencias. La grabación, la defensa, la revisión de simplificaciones con el profesor y las contribuciones personales siguen correspondiendo a los estudiantes.

Referencias

- 1. Sistemas Distribuidos 2026 2. Proyecto 1 DFSha. Guía de clase, 2026.
- 2. Ghemawat, S., Gobioff, H. y Leung, S. The Google File System. ACM SOSP, 2003.
- 3. Shvachko, K., Kuang, H., Radia, S. y Chansler, R. The Hadoop Distributed File System. IEEE MSST, 2010.
- 4. Ongaro, D. y Ousterhout, J. In Search of an Understandable Consensus Algorithm. USENIX ATC, 2014.
- 5. Gray, C. y Cheriton, D. Leases An Efficient Fault Tolerant Mechanism for Distributed File Cache Consistency. ACM SOSP, 1989.
- 6. IETF RFC 8446. The Transport Layer Security Protocol Version 1.3, 2018.
- 7. NIST SP 800 38D. Recommendation for Galois Counter Mode, 2007.